

Business requirement to evidence-backed release

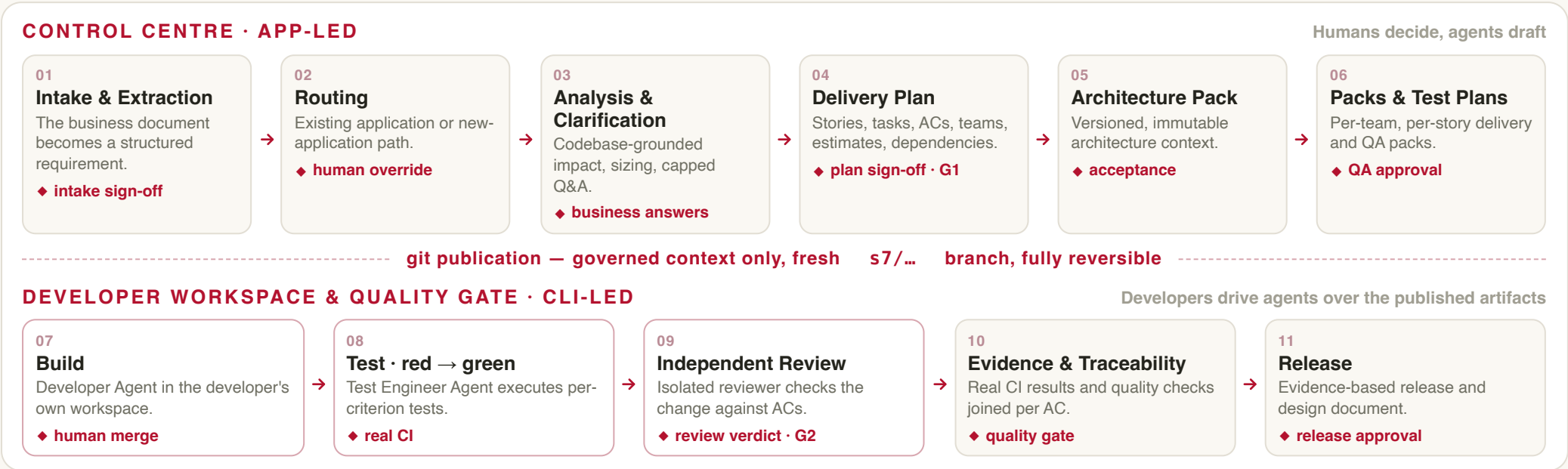
S7 is the governed control plane for an AI-assisted SDLC. Humans own implementation; S7 generates governed context, publishes it, collects real delivery evidence and orchestrates independent review.

Human controlled
Named business, engineering and QA gates prevent any phase from self-approving. Merging is always a human act.

Grounded in the estate
Every AI call is grounded in the connected repositories and each application's own architecture context — a file in the repo, not a fine-tune.

Evidence backed
Acceptance criteria, tests, real CI results, reviewer findings and approvals are joined into the release record — measured, never claimed.

1 THE PIPELINE AT A GLANCE



Separate roles, separate guardrails, one gateway

Each agent has a bounded responsibility and its own prompt. The operating model separates intake and planning, developer execution, and independent review — so no role grades its own work.

A UPSTREAM · CONTROL CENTRE

**Requirement Extraction Agent**

Turns .txt / .md / .pdf / .docx input into a structured requirement: title, objective, summary, numbered requirement bullets.

**Requirement Routing Agent**

Chooses an already-onboarded application or a new-application path. A human can override the verdict.

**Intake Analysis Agent**

Analyzes the epic against the connected repositories, extracts business rules, sizes the change, raises its own clarification questions.

**Clarification Agent**

Runs a capped Q&A with the business owner. Questions come from the analysis; the recorded answers feed the plan.

**New-Application Setup Agent**

Produces a reviewable architecture + README scaffold for human approval before any repository is created.

**Delivery Planning Agent**

Creates stories, tasks, acceptance criteria, team assignments, estimates and dependencies — with one bounded self-correction pass before a human sees the plan.

**Architecture Refinement Agent**

Refines a lead's proposed architecture change into a new immutable version. Human acceptance always resets.

**Test-Plan Refinement Agent**

Refines a QA lead's proposed amendment into additional cases as an overlay, then returns the pack to QA approval.

B DOWNSTREAM · DEVELOPER WORKSPACE & INDEPENDENT REVIEW

**Developer Agent**

Implements the task in the developer's own CLI workspace, grounded in **AGENTS.md**, acceptance criteria and approved architecture context. Reviewer findings triage back to it — never a blind retry.

**Test Engineer Agent**

Authors and runs the change's tests from the deliberately failing per-AC skeletons — so the evidence is a real red-to-green transition, per criterion.

**Independent Reviewer Agent**

Isolated from developer and tester roles. Reviews the finished change against the acceptance criteria — not the tests — issues the G2 verdict and returns named findings for revision.

DELIBERATELY NOT AGENTS
Deterministic services that cannot invent results

Architecture-pack assembly, delivery packs, rule-derived test skeletons, git publication, dependency wave gating, CI evidence joins, staleness tracking, quality checks and the release document are computed from real run data.

-  Bounded revision loops
-  Role isolation
-  Audited gateway
-  Visible provenance
-  Human-controlled merge

ONE AUDITED GATEWAY — every agent call runs through a single LLM module with per-call telemetry, pointed at any OpenAI-compatible endpoint: CL's enterprise gateway or a self-hosted model.

Human gates make the decisions; deterministic checks judge the evidence

The quality gate combines an isolated reviewer verdict with nine checks computed from the run's real evidence. A check that is arithmetic over evidence cannot hallucinate — which is why it sits in judgment over the agents.

1 NAMED HUMAN GATES

- ◆ Intake sign-off

Business Owner — the requirement is the business's artifact
- ◆ Plan sign-off · G1

Delivery Lead — authorises downstream, never performs it
- ◆ Architecture acceptance

Engineering Lead — every revision resets acceptance
- ◆ Test-plan approval

QA Lead — an unapproved pack cannot publish
- ◆ Publication

Explicit action — governed context, disposable branch
- ◆ Workspace assignment

Every story assigned to a developer by a human
- ◆ Dependency override

Lead only — the one early unlock, reason recorded
- ◆ Independent review · G2

Isolated reviewer verdict on every completed task
- ◆ Release approval

Human decision on the evidence-backed release document

2 AUTOMATED QUALITY VERIFICATION

- QC-01

Req → story mapping ✓

every story traces to the business requirement
- QC-02

AC → code mapping ✓

every criterion covered by a completed task
- QC-03

AC → test mapping ✓

every criterion carries a test
- QC-04

Test execution ✓

all tests passing in the target repository's CI
- QC-05

Code coverage ✓

measured coverage meets the agreed threshold
- QC-06

Security scan ✓

no untracked critical or major findings
- QC-07

Review gaps ✓

zero unresolved major reviewer findings
- QC-09

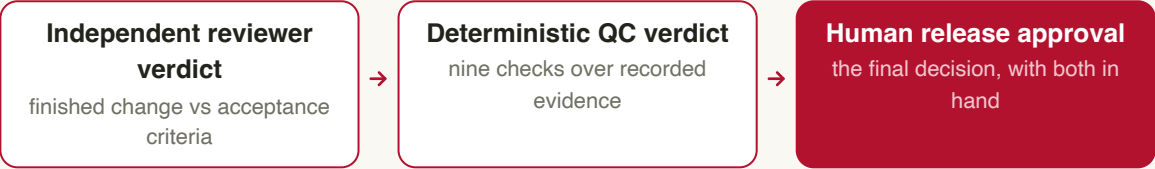
Dependency scan ✓

no known-vulnerable dependencies
- QC-10

Standards check ✓

engineering standards checks all green

3 DECISION FLOW



TRACEABILITY PRINCIPLE — every artifact carries visible provenance (AI, rule-based or human) and a chain back to the requirement that caused it. The S7 control plane itself never writes production code into CL's applications.

A server-validated phase machine from intake to release. Out-of-order actions are refused; each gate requires its named artifacts and sign-off.

Stable rules, role prompts, reference context and task delta assembled in fixed order, driven by one orchestrator.

Purpose, testable ACs, dependencies, owning team and estimate enforced at planning time — one bounded self-correction pass repairs defective drafts.

Every artifact records what produced it, when, and which upstream artifact it derives from. Revisions create new immutable versions.

Per-call telemetry including cache read/write tokens; an approvals ledger records every gate decision and who took it.

An isolated reviewer verifies acceptance criteria against the finished change. Critical or major gaps block and route back by role.

Acceptance criteria carry IDs
traced requirement → story →
task → test → CI result; the
release document renders each
criterion with its evidence.

Propose → refine → re-approve
loops for architecture and test
plans. Every change creates a
new version and resets
acceptance.

Machine-checkable checks before human review: every business rule claimed by a story, every acceptance criterion testable and covered.

When upstream artifacts change, downstream packs, workspaces and evidence are marked stale and must be regenerated before release.

Governed orchestration, state, versions, evidence and approvals — one small VM, local files, no estate integration.

Build and test happen in the developer's own IDE / CLI / Git environment — Human Controlled · AI Assisted.

Where run data does not exist, S7 reports nothing rather than a guess — the same discipline for KPIs, checks and documents.

BUILT TO PORT — plain Python with pinned dependencies, local state, and deterministic replay of every rehearsed run: a fresh install with no API key still executes the full pipeline offline, which is how the system survives a locked-down environment.

S7 sits beside the estate; it does not replace the estate

The control plane reads approved repositories, publishes only governed context to disposable branches, consumes CI evidence, and routes every LLM call through the approved gateway or a self-hosted model.

1 INTEGRATION MODEL

CL ESTATE — UNTOUCHED

Existing engineering systems

- Application repositories — read access per onboarded application
- CI pipeline — per-test results become S7 evidence
- Enterprise LLM gateway or self-hosted model
- The humans holding the named gates

→
repo
read

←
context
branch

→
CI
results

S7 — ONE SMALL VM

Governed control plane + web UI

- All pipeline state in local files — no database integration with CL systems
- Plain Python — no Docker, no cloud-managed services, no CDN
- Built for a locked-down environment
- Per-application `architecture.md` grounds every AI call

↔
packs
out

←
PRs
back

DEVELOPER WORKSPACE

>_ Human controlled, AI assisted

- Developers' own IDE / CLI / Git
- Developer Agent implements against the published pack
- Test Engineer Agent runs the red-to-green tests
- Humans decide what is merged into working branches

SURGICAL PUBLICATION — only `AGENTS.md`, `.s7/**` and the governed test root, on a fresh `s7/...` branch per run.

PROTECTED DEFAULTS — S7 refuses to touch a default branch and refuses to overwrite content it did not write.

FULLY REVERSIBLE — delete the disposable context branch and nothing remains in the application estate.

2 WHAT WE NEED FROM CL

- 1 **Approved LLM endpoint + keys** — their gateway or ours, their call.
- 2 **Repository access** — read per onboarded application, plus context-branch push (or export-only mode with no write access at all).
- 3 **CI visibility** — per-test results, or permission to add the bootstrapped workflow, so build/test evidence is real, not claimed.
- 4 **Named gate owners** — business owner, delivery/engineering lead and QA lead per project. Non-delegable by design.
- 5 **Architecture context sign-off** — per-application approval of the `architecture.md` that grounds every AI call.

3 COST BUCKETS

- 1 **LLM consumption** — the main variable cost, metered per run; prompt-prefix ordering maximises provider-side caching, and cache tokens are logged per call, so cost per release is measured, not estimated.
- 2 **Hosting** — one small VM.
- 3 **Onboarding per application** — repository connection + context pack, one-off.
- 4 **Ongoing** — gate-keeper time (deliberately human-owned; the gates are the product, not overhead) plus licence/support under the commercial model.

End-to-end coverage, visible in artifacts, evidence and sign-offs

S7's claim is coverage of the SDLC's deliverables, not depth in any one of them: each phase produces a named, versioned artifact with visible provenance, and that artifact is the input to the next phase's gate.

1 Requirement Structured requirement from the business document, plus the extracted business-rule set (AI + human-added rules). SIGN-OFF — Business Owner intake gate.	2 Analysis Codebase-grounded impact analysis, sizing, and the clarification Q&A transcript. EVIDENCE — business answers recorded on the run.	3 Planning Stories, tasks, acceptance-criteria ids, teams, estimates and the dependency map with delivery waves. SIGN-OFF — completeness checks, then plan sign-off (G1).	4 Design Versioned architecture pack, refined through propose → refine → re-approve. SIGN-OFF — Engineering Lead acceptance, reset on each revision.
5 Build Per-team, per-story delivery packs; developer workspaces; implementation in CL's own git. EVIDENCE — dependency-gated start, commits and PRs.	6 Test QA-approved plan per story; per-acceptance-criterion tests taken from red to green. EVIDENCE — real CI results joined to criteria by test name.	7 Review Independent review verdict per completed task, with findings triaged back by role. SIGN-OFF — review gate (G2) plus the nine QC checks.	8 Release Release/design document rendering approvals, per-story verdicts and every criterion with its result. SIGN-OFF — human release approval, evidence-backed.

K DELIVERY KPIS PRODUCED FROM RUN DATA

- velocity
- cycle time
- estimation accuracy
- defect leakage
- first-time-right rate
- on-time / on-budget delivery
- cost per release

IP & DATA BOUNDARY

🔒 Ownership boundary The control plane and tooling are our IP. Plans, architecture packs, delivery packs, test plans, evidence records and release documents it produces are CL's.	🛡️ Model boundary The LLM is accessed under enterprise terms with no training on CL data; the gateway / self-hosted option keeps CL code and data inside their boundary.	🔗 Estate boundary The only content written into CL's estate is governed context on disposable branches that a CL human chooses to merge — or not.
--	--	---

MEASUREMENT DISCIPLINE — every gate decision, agent session and LLM call is logged (including cache read/write tokens); each KPI traces to the records behind it. Where the data does not exist yet, S7 reports nothing rather than a guess.